

```
# import shutil
# shutil.rmtree('/content/bace_out')
```

```
!git clone https://github.com/ChemFoundationModels/ChemLLMBench.git
```

```
Cloning into 'ChemLLMBench'...
remote: Enumerating objects: 244, done.
remote: Counting objects: 100% (157/157), done.
remote: Compressing objects: 100% (108/108), done.
remote: Total 244 (delta 78), reused 96 (delta 43), pack-reused 87 (from 1)
Receiving objects: 100% (244/244), 4.27 MiB | 10.53 MiB/s, done.
Resolving deltas: 100% (103/103), done.
```

```
!pip install -U bitsandbytes
```

```
Collecting bitsandbytes
  Downloading bitsandbytes-0.49.0-py3-none-manylinux_2_24_x86_64.whl.metadata (10 kB)
Requirement already satisfied: torch<3,>=2.3 in /usr/local/lib/python3.12/dist-packages (from bitsandbytes) (2.9.0+cu126)
Requirement already satisfied: numpy>=1.17 in /usr/local/lib/python3.12/dist-packages (from bitsandbytes) (2.0.2)
Requirement already satisfied: packaging>=20.9 in /usr/local/lib/python3.12/dist-packages (from bitsandbytes) (25.0)
Requirement already satisfied: filelock in /usr/local/lib/python3.12/dist-packages (from torch<3,>=2.3->bitsandbytes) (3.20.0)
Requirement already satisfied: typing-extensions>=4.10.0 in /usr/local/lib/python3.12/dist-packages (from torch<3,>=2.3->bitsandbytes) (4.10.0)
Requirement already satisfied: setuptools in /usr/local/lib/python3.12/dist-packages (from torch<3,>=2.3->bitsandbytes) (75.2.0)
Requirement already satisfied: sympy>=1.13.3 in /usr/local/lib/python3.12/dist-packages (from torch<3,>=2.3->bitsandbytes) (1.13.3)
Requirement already satisfied: networkx>=2.5.1 in /usr/local/lib/python3.12/dist-packages (from torch<3,>=2.3->bitsandbytes) (3.3)
Requirement already satisfied: Jinja2 in /usr/local/lib/python3.12/dist-packages (from torch<3,>=2.3->bitsandbytes) (3.1.6)
Requirement already satisfied: fsspec>=0.8.5 in /usr/local/lib/python3.12/dist-packages (from torch<3,>=2.3->bitsandbytes) (2024.10.0)
Requirement already satisfied: nvidia-cuda-nvrtc-cu12==12.6.77 in /usr/local/lib/python3.12/dist-packages (from torch<3,>=2.3->bitsandbytes) (12.6.77)
Requirement already satisfied: nvidia-cuda-runtime-cu12==12.6.77 in /usr/local/lib/python3.12/dist-packages (from torch<3,>=2.3->bitsandbytes) (12.6.77)
Requirement already satisfied: nvidia-cuda-cupti-cu12==12.6.80 in /usr/local/lib/python3.12/dist-packages (from torch<3,>=2.3->bitsandbytes) (12.6.80)
Requirement already satisfied: nvidia-cudnn-cu12==9.10.2.21 in /usr/local/lib/python3.12/dist-packages (from torch<3,>=2.3->bitsandbytes) (9.10.2.21)
Requirement already satisfied: nvidia-cublas-cu12==12.6.4.1 in /usr/local/lib/python3.12/dist-packages (from torch<3,>=2.3->bitsandbytes) (12.6.4.1)
Requirement already satisfied: nvidia-cufft-cu12==11.3.0.4 in /usr/local/lib/python3.12/dist-packages (from torch<3,>=2.3->bitsandbytes) (11.3.0.4)
Requirement already satisfied: nvidia-curand-cu12==10.3.7.77 in /usr/local/lib/python3.12/dist-packages (from torch<3,>=2.3->bitsandbytes) (10.3.7.77)
Requirement already satisfied: nvidia-cusolver-cu12==11.7.1.2 in /usr/local/lib/python3.12/dist-packages (from torch<3,>=2.3->bitsandbytes) (11.7.1.2)
Requirement already satisfied: nvidia-cusparselt-cu12==0.7.1 in /usr/local/lib/python3.12/dist-packages (from torch<3,>=2.3->bitsandbytes) (0.7.1)
Requirement already satisfied: nvidia-nccl-cu12==2.27.5 in /usr/local/lib/python3.12/dist-packages (from torch<3,>=2.3->bitsandbytes) (2.27.5)
Requirement already satisfied: nvidia-nvshmem-cu12==3.3.20 in /usr/local/lib/python3.12/dist-packages (from torch<3,>=2.3->bitsandbytes) (3.3.20)
Requirement already satisfied: nvidia-nvtx-cu12==12.6.77 in /usr/local/lib/python3.12/dist-packages (from torch<3,>=2.3->bitsandbytes) (12.6.77)
Requirement already satisfied: nvidia-nvjitlink-cu12==12.6.85 in /usr/local/lib/python3.12/dist-packages (from torch<3,>=2.3->bitsandbytes) (12.6.85)
Requirement already satisfied: nvidia-cufile-cu12==1.11.1.6 in /usr/local/lib/python3.12/dist-packages (from torch<3,>=2.3->bitsandbytes) (1.11.1.6)
Requirement already satisfied: triton==3.5.0 in /usr/local/lib/python3.12/dist-packages (from torch<3,>=2.3->bitsandbytes) (3.5.0)
Requirement already satisfied: mpmath<1.4,>=1.1.0 in /usr/local/lib/python3.12/dist-packages (from sympy>=1.13.3->torch<3,>=2.3->bitsandbytes) (1.3.0)
Requirement already satisfied: MarkupSafe>=2.0 in /usr/local/lib/python3.12/dist-packages (from Jinja2->torch<3,>=2.3->bitsandbytes) (3.0.2)
Downloading bitsandbytes-0.49.0-py3-none-manylinux_2_24_x86_64.whl (59.1 MB)
59.1/59.1 MB 9.9 MB/s eta 0:00:00
Installing collected packages: bitsandbytes
Successfully installed bitsandbytes-0.49.0
```

```
import bitsandbytes
import torch
from transformers import AutoModelForCausalLM, AutoTokenizer, BitsAndBytesConfig
from peft import LoraConfig, get_peft_model, prepare_model_for_kbit_training

model_name = "meta-llama/Llama-3.2-3B-Instruct" # or any HF model you can access

bnb_config = BitsAndBytesConfig(
    load_in_4bit=True,
    bnb_4bit_compute_dtype=torch.float16,
    bnb_4bit_quant_type="nf4",
    bnb_4bit_use_double_quant=True,
)

tokenizer = AutoTokenizer.from_pretrained(model_name, use_fast=True)
if tokenizer.pad_token is None:
    tokenizer.pad_token = tokenizer.eos_token

model = AutoModelForCausalLM.from_pretrained(
    model_name,
    quantization_config=bnb_config,
    device_map="auto",
)
```

```
model = prepare_model_for_kbit_training(model)

lora_cfg = LoraConfig(
    r=8,
    lora_alpha=16,
    lora_dropout=0.0,
    bias="none",
    task_type="CAUSAL_LM",
    target_modules=["q_proj", "k_proj", "v_proj", "o_proj", "gate_proj", "up_proj", "down_proj"],
)

model = get_peft_model(model, lora_cfg)
```

Loading checkpoint shards: 100%

2/2 [00:43<00:00, 19.44s/it]

```
import pandas as pd
from datasets import Dataset

# Path to your training CSV
train_csv = "/content/ChemLLMBench/data/property_prediction/BACE.csv" # adjust path

df = pd.read_csv(train_csv)

# Detect SMILES column name safely
smiles_col = "mol" # Explicitly set to 'mol' as per DataFrame contents
label_col = "Class"
```

```
def make_prompt(row):
    return (
        "Instruction:"
        "You are an expert chemist specializing in molecular property prediction."
        "Given a molecule's SMILES string, determine whether the compound can inhibit Beta-site Amyloid Precursor Protein C1"
        "Base your judgment on structural features such as molecular weight, atom composition, bond types, functional groups"
        "Respond with only one word – 1 if the molecule can inhibit BACE1; implying Yes, or 0 if it cannot; implying No."
        "Do not provide any explanation or additional text.\n"
        f"Input:{row[smiles_col]}\n"
        f"Output:{(row[label_col])}"
    )

df["text"] = df.apply(make_prompt, axis=1)
```

df

	mol	CID	Class	Model	pIC50	MW	AlogP	HBA	HBD	RB	...	(P
0	<chem>O1CC[C@@H](NC(=O)[C@@H](Cc2cc3cc(ccc3nc2N)-c2c...</chem>	BACE_1	1	Train	9.154901	431.56979	4.4014	3	2	5	...	78.6
1	<chem>Fc1cc(cc(F)c1)C[C@H](NC(=O)[C@@H](N1CC[C@](NC(...</chem>	BACE_2	1	Train	8.853872	657.81073	2.6412	5	4	16	...	47.1
2	<chem>S1(=O)(=O)N(c2cc(cc3c2n(cc3CC)CC1)C(=O)N[C@H](...</chem>	BACE_3	1	Train	8.698970	591.74091	2.5499	4	3	11	...	47.9
3	<chem>S1(=O)(=O)C[C@H](Cc2cc(O[C@H](COCC)C(F)(F)F)c...</chem>	BACE_4	1	Train	8.698970	591.67828	3.1680	4	3	12	...	37.9
4	<chem>S1(=O)(=O)N(c2cc(cc3c2n(cc3CC)CC1)C(=O)N[C@H](...</chem>	BACE_5	1	Train	8.698970	629.71283	3.5086	3	3	11	...	39.3
...	...	...	...	...	...	...	...	...	...	...	...	...
1508	<chem>Clc1cc2nc(n(c2cc1)C(CC(=O)NCC1CCOCC1)CC)N</chem>	BACE_1543	0	Test	3.000000	364.86969	2.5942	3	2	6	...	37.6
1509	<chem>Clc1cc2nc(n(c2cc1)C(CC(=O)NCc1cccc1)CC)N</chem>	BACE_1544	0	Test	3.000000	357.83731	2.8229	3	2	6	...	47.3
1510	<chem>Brc1cc(ccc1)C1CC1C=1N=C(N)N(C)C(=O)C=1</chem>	BACE_1545	0	Test	2.953115	320.18451	3.0895	2	1	2	...	22.5
1511	<chem>O=C1N(C)C(=NC(=C1)C1CC1c1cc(ccc1)-c1cccc1)N</chem>	BACE_1546	0	Test	2.733298	317.38440	3.8595	2	1	3	...	9.3
1512	<chem>Clc1cc2nc(n(c2cc1)CCCC(=O)NCC1CC1)N</chem>	BACE_1547	0	Test	2.544546	306.79059	3.4271	2	2	6	...	37.6

1513 rows × 596 columns

```
ds_train = Dataset.from_pandas(df[["text"]])
```

```
display(ds_train[1])
```

```
{'text': 'Instruction:You are an expert chemist specializing in molecular property prediction.Given a molecule’s SMILES string, determine whether the compound can inhibit Beta-site Amyloid Precursor Protein Cleaving Enzyme 1 (BACE1).Base your judgment on structural features such as molecular weight, atom composition, bond types, functional groups, and overall drug-likeness relevant to Alzheimer’s disease therapeutics.Respond with only one word – 1 if the molecule can inhibit BACE1; implying Yes, or 0 if it cannot; implying No.Do not provide any explanation or additional text.\nInput:Fc1cc(cc(F)c1)C[C@H](NC(=O)[C@@H](N1CC[C@](NC(=O)C)(CC(C)C)C1=O)CCc1cccc1)[C@H](O)[C@@H]1[NH2+][C[C@H](OCCC)C1\nOutput:1'}
```

```
def tokenize_fn(examples):
    tokens = tokenizer(
        examples["text"],
        truncation=True,
        max_length=512,
        padding="max_length",
    )
    tokens["labels"] = tokens["input_ids"].copy()
    return tokens
```

```
ds_train_tok = ds_train.map(
    tokenize_fn,
    batched=True,
    remove_columns=ds_train.column_names,
)
```

Map: 100%

1513/1513 [00:01<00:00, 962.77 examples/s]

```
from transformers import Trainer, TrainingArguments, DataCollatorForLanguageModeling
```

```
collator = DataCollatorForLanguageModeling(
    tokenizer=tokenizer,
    mlm=False,
)
```

```
training_args = TrainingArguments(
    output_dir="bace_out",
    per_device_train_batch_size=1,
    gradient_accumulation_steps=32,
    learning_rate=2e-4,
    num_train_epochs=1,
    fp16=True,
    logging_steps=10,
    save_steps=200,
    report_to="none",
)
```

```
trainer = Trainer(
    model=model,
    args=training_args,
    train_dataset=ds_train_tok,
    data_collator=collator,
)
```

```
trainer.train()
```

`use\_cache=True` is incompatible with gradient checkpointing. Setting `use\_cache=False`.
 /usr/local/lib/python3.12/dist-packages/torch/_dynamo/eval_frame.py:1044: UserWarning: torch.utils.checkpoint: the use_reentrant return fn(*args, **kwargs)

 [48/48 24:11, Epoch 1/1]

Step	Training Loss
------	---------------

10	1.701800
20	0.480400
30	0.406500
40	0.349500

```
TrainOutput(global_step=48, training_loss=0.6671266357103983, metrics={'train_runtime': 1491.6344,
'train_samples_per_second': 1.014, 'train_steps_per_second': 0.032, 'total_flos': 1.315786210148352e+16, 'train_loss':
0.6671266357103983, 'epoch': 1.0})
```

```
model.save_pretrained("bace_lora_adapters")
tokenizer.save_pretrained("bace_lora_adapters")
```

```
('bace_lora_adapters/tokenizer_config.json',
'bace_lora_adapters/special_tokens_map.json',
'bace_lora_adapters/chat_template.jinja',
'bace_lora_adapters/tokenizer.json')
```

```
!zip -r /content/bace_lora_adapters.zip /content/bace_lora_adapters
```

```
adding: content/bace_lora_adapters/ (stored 0%)
adding: content/bace_lora_adapters/adapters_model.safetensors (deflated 8%)
adding: content/bace_lora_adapters/special_tokens_map.json (deflated 63%)
adding: content/bace_lora_adapters/adapters_config.json (deflated 58%)
adding: content/bace_lora_adapters/tokenizer_config.json (deflated 96%)
adding: content/bace_lora_adapters/chat_template.jinja (deflated 71%)
adding: content/bace_lora_adapters/README.md (deflated 65%)
adding: content/bace_lora_adapters/tokenizer.json (deflated 85%)
```

```
import torch
from transformers import AutoModelForCausalLM, AutoTokenizer # Already imported, but good for standalone
from peft import PeftModel # Ensure PeftModel is available for merge_and_unload

# The 'model' in the current kernel state is already a PeftModel after training.
# We need to merge the adapters to the base model for inference.
# Ensure the model is in evaluation mode
model.eval()

# Merge the LORA adapters with the base model.
# This will unload the adapters and return a fully merged model.
# The 'model' variable will now hold the merged model.
model = model.merge_and_unload()

# Ensure tokenizer padding_side is set to 'left' for inference
tokenizer.padding_side = "left"
```

```
/usr/local/lib/python3.12/dist-packages/peft/tuners/lora/bnb.py:397: UserWarning: Merge lora module to 4-bit linear may get d
warnings.warn(
```

```
# Example inference using a sample SMILES string
test_smiles = pd.read_csv("/content/ChemLLMBench/data/property_prediction/BACE_test.csv")
X_test, Y_test = test_smiles['mol'], test_smiles['Class']
```

X_test

	mol
0	S1(=O)(=O)NCC2(N1c1cc(F)ccc1)CC([NH+](CC2)Cc1c...
1	S1(=O)(=O)N(c2cc(cc3n(cc(CC1)c23)CC)C(=O)NC([C...
2	O(C(=O)C1[NH2+]CC2(C1)c1c(NC2=O)ccc1)C1CCN(CC...
3	Clc1cc(cnc1)-c1cc2c(CC(CC23N=C(OC3)N)(C)C)cc1
4	Clc1cc(cc(Cl)c1NC(=O)C)CNC(=[NH2+])NC(=O)Cn1c2...
...	...
95	OC(C(NC(=O)c1c2cccnc2n(c1)C(=O)N(CCCC)C)Cc1ccc...
96	FC(F)Oc1ccc(cc1)[C@@]1(N=C(N)N(C)C1=O)c1cc(ccc...
97	S(=O)(=O)(N(C)c1cc2cc(c1)C(=O)NCCCCOCC(NC2=O)...
98	S1(=O)(=O)N(c2cc(cc3c2n(cc3CC)CC1)C(=O)N[C@H](...
99	Fc1cc(cc(F)c1)CC(NC(=O)C)C(O)C[NH2+]C1(CCCCC1)...

100 rows x 1 columns

dtype: object

```
def create_prompt(smiles):
    # The instruction part of the prompt should match the training prompt structure
    prompt_instruction = (
        "Instruction:"
        "You are an expert chemist specializing in molecular property prediction."
        "Given a molecule's SMILES string, determine whether the compound can inhibit Beta-site Amyloid Precursor Protein C1"
        "Base your judgment on structural features such as molecular weight, atom composition, bond types, functional groups"
        "Respond with only one word - 1 if the molecule can inhibit BACE1; implying Yes, or 0 if it cannot; implying No."
        "Do not provide any explanation or additional text.\n"
    )

    full_prompt = f"{prompt_instruction}\n### Input:\n{smiles}\n\n### Output:"
    return full_prompt
```

```
def generate_output(prompt):
    inputs = tokenizer([prompt], return_tensors="pt").to("cuda")

    # Generate response
    # max_new_tokens should be small since the expected output is 'Yes' or 'No'
    outputs = model.generate(
        input_ids=inputs["input_ids"],
```

```

attention_mask=inputs["attention_mask"],
max_new_tokens=5, # 'Yes' or 'No' plus potential newline/token
do_sample=True,
top_p=0.9, # To encourage more deterministic output for classification
temperature=0.7, # Lower temperature for more focused output
pad_token_id=tokenizer.pad_token_id
)

# Decode the generated output
generated_text = tokenizer.decode(outputs[0], skip_special_tokens=True)
# print(f"Full generated text:\n{generated_text}")

# Extract just the predicted answer (e.g., 'Yes' or 'No')
try:
    predicted_answer = generated_text.split("### Output:")[-1].strip()
    if "Yes" in predicted_answer:
        predicted_answer = "Yes"
    elif "No" in predicted_answer:
        predicted_answer = "No"
    else:
        # Fallback if the output format is not strictly "Yes" or "No" alone
        predicted_answer = predicted_answer.split('\n')[0].strip()

    return predicted_answer
except:
    return "Error"

```

```

responses = []
for smile in X_test:
    prompt = create_prompt(smile)
    response = generate_output(prompt)
    responses.append(response)

```

```

from sklearn.metrics import accuracy_score, precision_score, recall_score, f1_score
import numpy as np

# Convert 'Yes'/'No' responses to 1/0
predicted_labels = []
for r in responses:
    if r == 'Yes' or r == '1':
        predicted_labels.append(1)
    elif r == 'No' or r == '0':
        predicted_labels.append(0)
    else:
        # Handle cases where response might not be 'Yes' or 'No'
        predicted_labels.append(np.nan) # Or some other suitable handling

# Convert Y_test to a list for easier comparison
true_labels = Y_test.tolist()

```

```

# Count the occurrences of 0s and 1s in the predicted labels
label_counts = pd.Series(predicted_labels).value_counts().sort_index()

print("Count of predicted labels (0s and 1s):")
print(label_counts)

```

```

Count of predicted labels (0s and 1s):
0    12
1    88
Name: count, dtype: int64

```

```

# Count the occurrences of 0s and 1s in the predicted labels
label_counts = pd.Series(true_labels).value_counts().sort_index()

print("Count of true labels (0s and 1s):")
print(label_counts)

```

```

Count of true labels (0s and 1s):
0    64
1    36
Name: count, dtype: int64

```

```

# Filter out NaN values if any (from invalid responses)
# This assumes we are only evaluating valid predictions

```

```
# This assumes we are only evaluating valid predictions
valid_indices = [i for i, x in enumerate(predicted_labels) if not np.isnan(x)]
true_labels_filtered = [true_labels[i] for i in valid_indices]
predicted_labels_filtered = [int(predicted_labels[i]) for i in valid_indices]

# Calculate metrics
accuracy = accuracy_score(true_labels_filtered, predicted_labels_filtered)
precision = precision_score(true_labels_filtered, predicted_labels_filtered)
recall = recall_score(true_labels_filtered, predicted_labels_filtered)
f1 = f1_score(true_labels_filtered, predicted_labels_filtered)

print(f"Accuracy: {accuracy:.4f}")
print(f"Precision: {precision:.4f}")
print(f"Recall: {recall:.4f}")
print(f"F1-Score: {f1:.4f}")
```

```
Accuracy: 0.3600
Precision: 0.3409
Recall: 0.8333
F1-Score: 0.4839
```

```
import pandas as pd

results_df = pd.DataFrame({
    'True_Label': true_labels_filtered,
    'Predicted_Label': predicted_labels_filtered
})

results_df
```

1 to 25 of 100 entries Filter  

index	True_Label	Predicted_Label
0	0	1
1	1	1
2	0	1
3	0	1
4	1	1
5	0	1
6	0	1
7	0	1
8	0	1
9	1	1
10	1	1
11	0	1